조수연 교수

공학생을위한실용AI

5주차. 프롬프트 엔지니어링

2강. 효과적인 프롬프트 전략



영진전문대학교에서 수업 자료로 사용되는 (동영상, 교안) 저작물은『저작권법 제25조 학교교육 목적 등에의 이용』의거하여, 적법하게 이용하고 있습니다.
수업목적 이외의 사용은 저작권법에 저촉될 수 있으므로 수업자료(동영상, 교안)의 대중 공개 , 공유 , 복제 , 전송 등 수업목적 외의 사용을 금지합니다.

학습내용 및 학습목표

학습내용

1. 인-컨텍스트 학습
2. 생각의 사슬
3. 난제 분해 기법
4. 에이전틱 사고

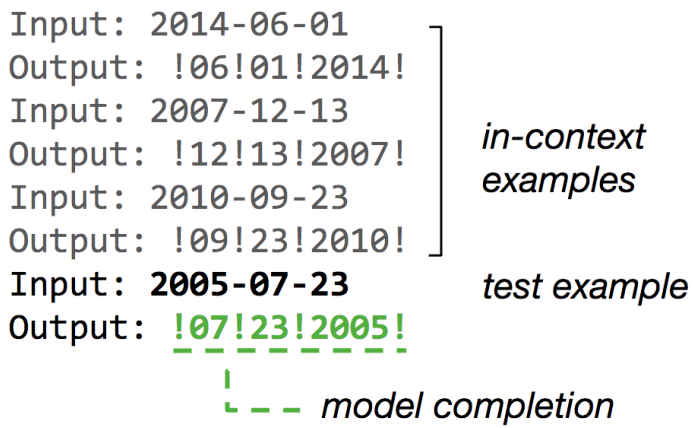
학습목표

1. 효과적인 프롬프팅을 할 수 있는 다양한 기법을 설명할 수 있다.

1. 인-컨텍스트 학습 (1)

1) 인-컨텍스트 학습

- In-context Learning
- 모델 내부의 파라미터를 수정하지 않고 맥락을 받아들이는 것
- 대화 도중에 제공된 예시를 보고 작업의 패턴을 파악



1. 인-컨텍스트 학습 (2)

2) 퓨샷 (Few-shot)

- 제로샷 프롬프팅: 예시 없이 바로 지시
- 퓨샷 프롬프팅: 3-5개의 예시 제공
  - 유사한 방법으로 원샷 프롬프팅도 있음

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← examples
3  peppermint => menthe poivrée        ←
4  plush girafe => girafe peluche      ←
5  cheese =>                           ← prompt
```

출처 01 The Stanford AI Lab - In-context Learning

1. 인-컨텍스트 학습 (3)

3) 실습 준비

- 생성형 AI 프롬프트 준비
- ChatGPT, Gemini 등 본인이 사용할 수 있는 생성형 AI
- 유료 버전 필요 없음
- 두 개 이상의 AI를 준비해서 결과를 비교해도 됨

1. 인-컨텍스트 학습 (4)

4) 실습 - 제로샷과 퓨샷

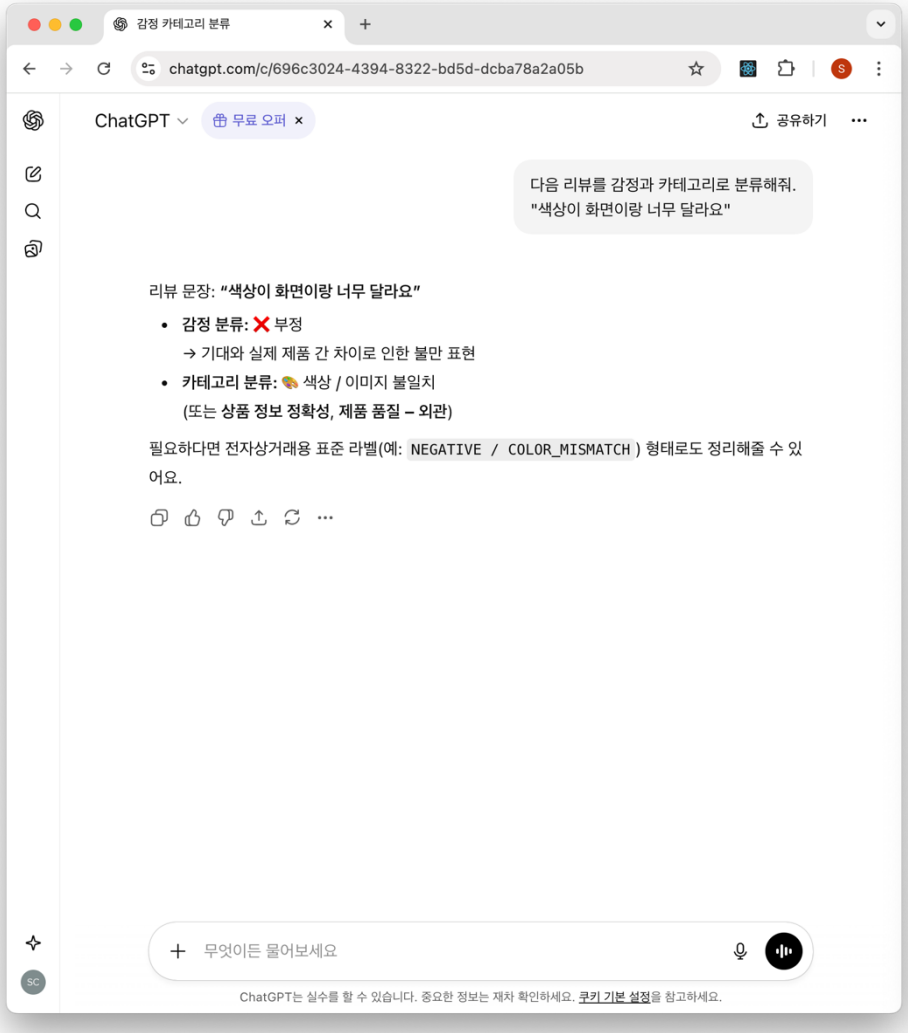
제로샷

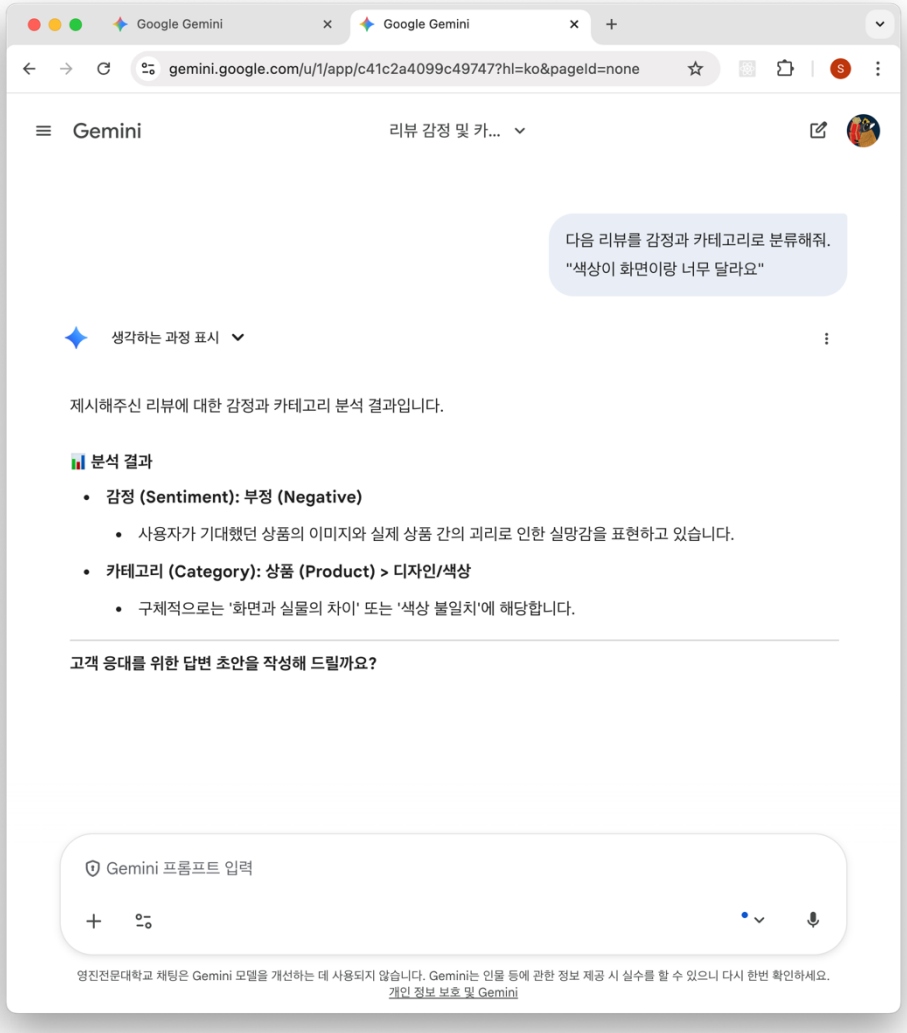
다음 리뷰를 감정과 카테고리로 분류해줘.
"색상이 화면이랑 너무 달라요"

- 출력 형식과 예시 없이 바로 질문

1. 인-컨텍스트 학습 (5)

5) 실습 - 제로샷 결과 예시





1. 인-컨텍스트 학습 (6)

6) 실습 - 제로샷과 퓨샷

퓨샷

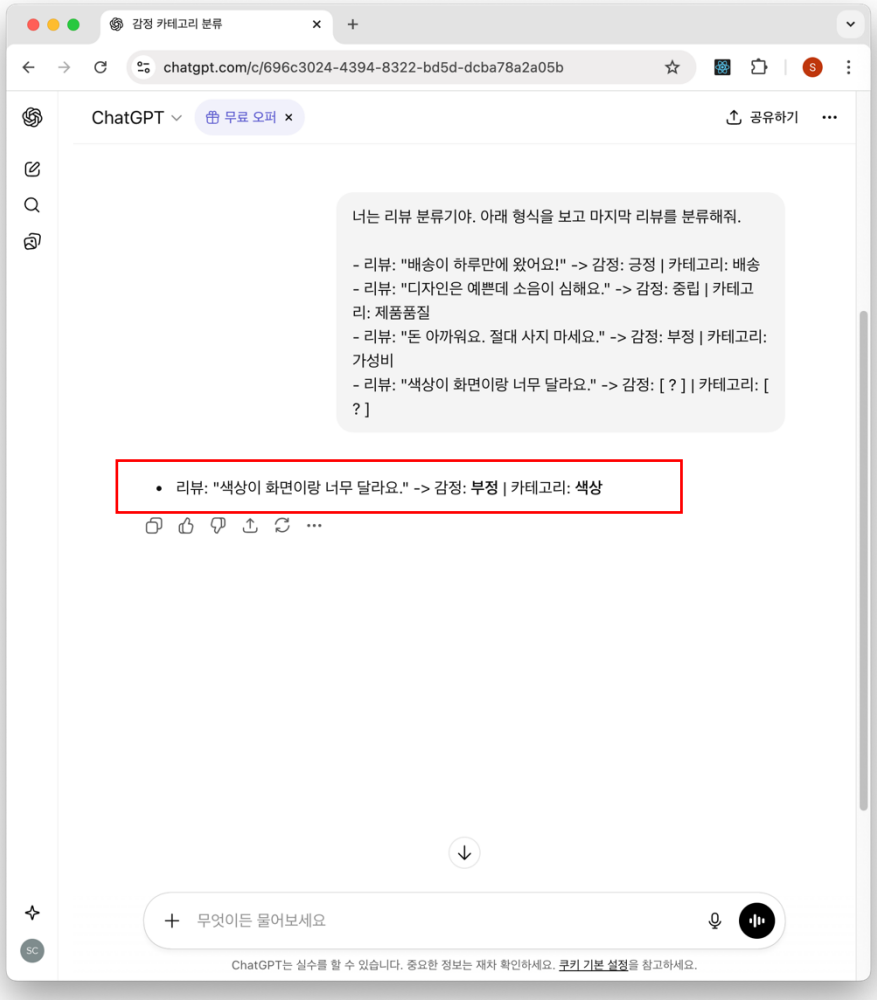
너는 리뷰 분류기야. 아래 형식을 보고 마지막 리뷰를 분류해줘.

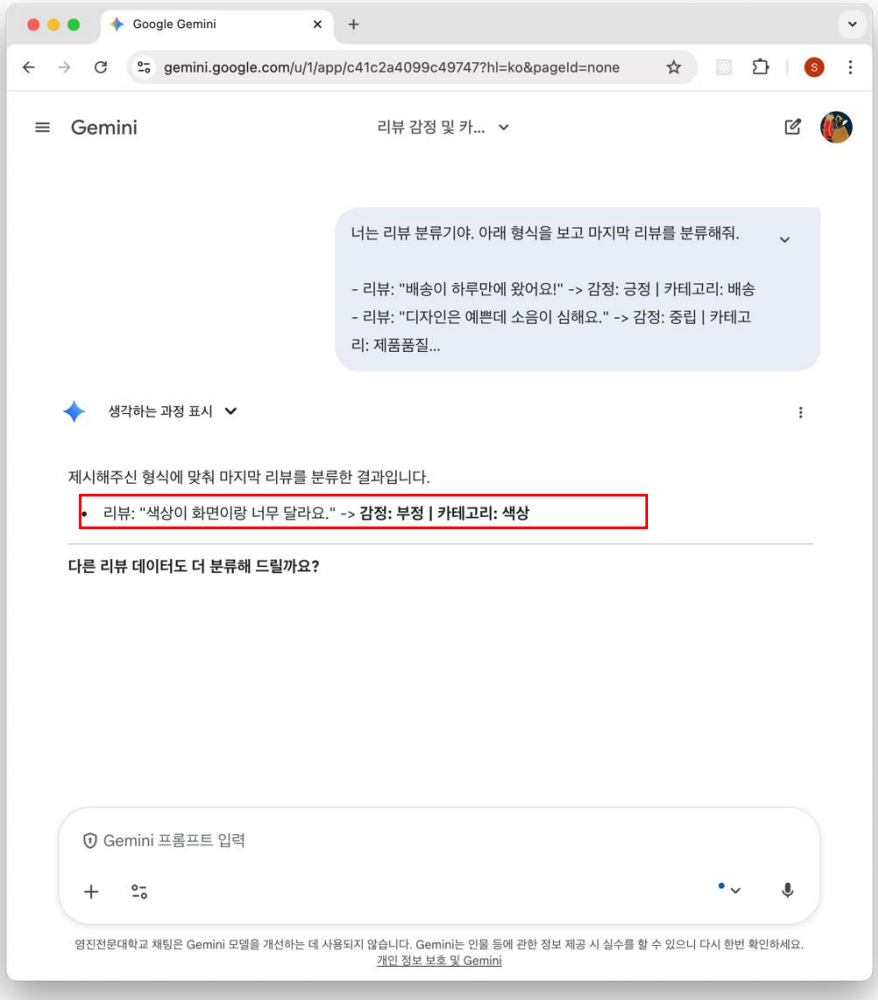
- 리뷰: "배송이 하루만에 왔어요!" -> 감정: 긍정 | 카테고리: 배송
- 리뷰: "디자인은 예쁜데 소음이 심해요." -> 감정: 중립 | 카테고리: 제품품질
- 리뷰: "돈 아까워요. 절대 사지 마세요." -> 감정: 부정 | 카테고리: 가성비
- 리뷰: "색상이 화면이랑 너무 달라요." -> 감정: [?] | 카테고리: [?]

- 원하는 출력 형식 지정.

1. 인-컨텍스트 학습 (7)

7) 실습 - 퓨샷 결과 예시





1. 인-컨텍스트 학습 (8)

8) 실습 - 퓨샷 활용 옛

퓨샷

너는 3년 차 스프링부트 개발자야.
다음 <code><code/>를 참조하여 CRUD 중 나머지 함수를 추가해줘.
customerService의 메소드 이름은 컨트롤러의 메소드 이름과 같게 해줘.

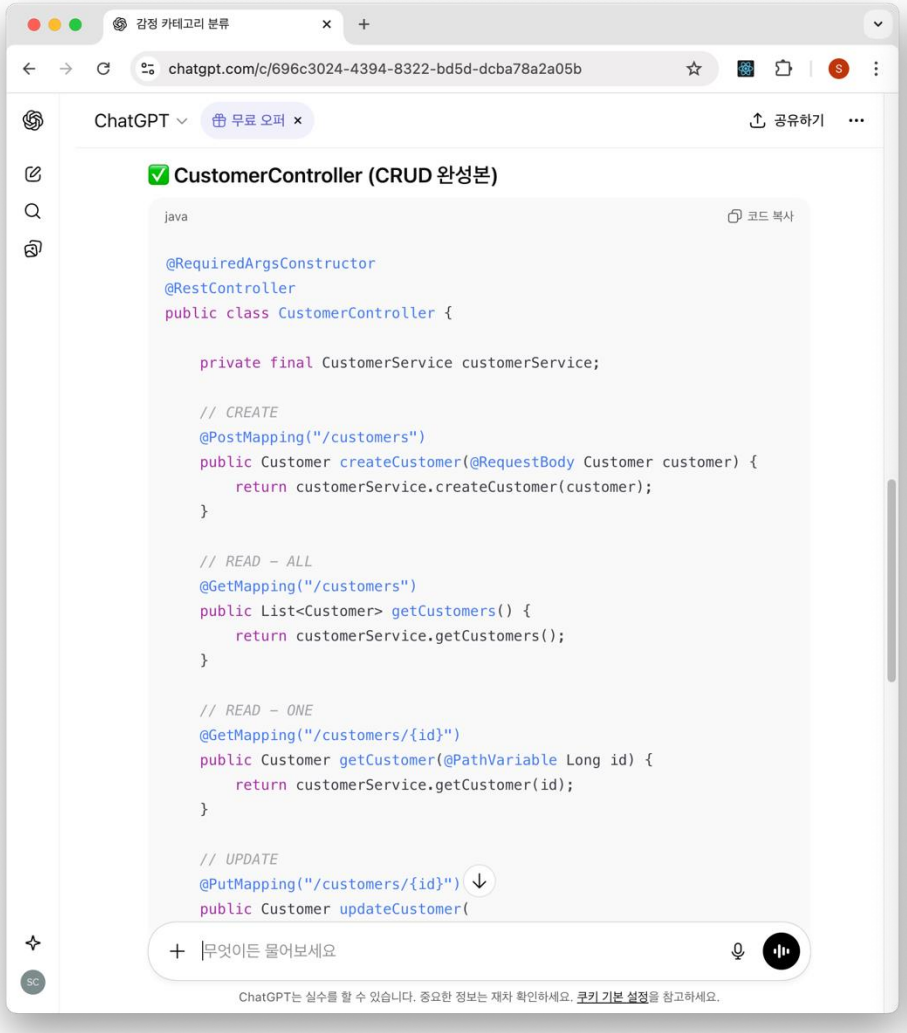
<code>
@RequiredArgsConstructor
@RestController
public class CustomerController{

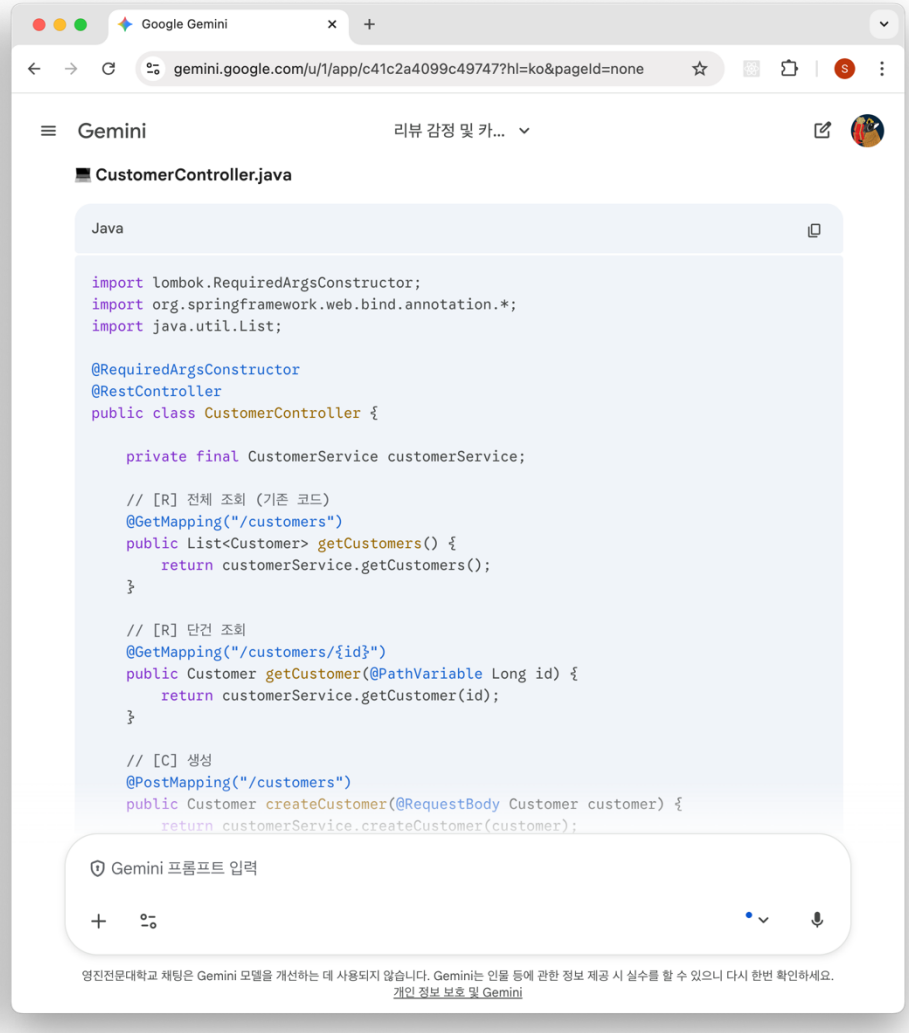
    private final CustomerService customerService;

    @GetMapping("/customers")
    public List<Customer> getCustomers(){
        return customerService.getCustomers();
    }
}
</code>

1. 인-컨텍스트 학습 (9)

9) 실습 - 퓨샷 활용 예시





2. 생각의 사슬 (1)

1) 생각의 사슬: Chain of Thought

- 복잡한 질문에 대해 정답을 바로 내기 전에 추론 과정을 생성하도록 유도하는 기술

- 특별한 예시 없이 "단계별로 생각하라"는 문구로 추론 성능이 올라감
- 오류를 범했을 때 다음 토큰 생성 과정에서 수정할 기회를 얻음

- 최신 모델은 사용자가 시키지 않아도 내부적으로 수천 개의 사고 경로를 검토함
  - 정답에 도달하는 논리의 단계에 보상을 받으며 훈련
  - AI 스스로 여러 경로로 문제를 풀어보고 오류가 나면 되돌아감
  - 최신 모델에서 답변까지 몇 초 걸리는 이유

2. 생각의 사슬 (2)

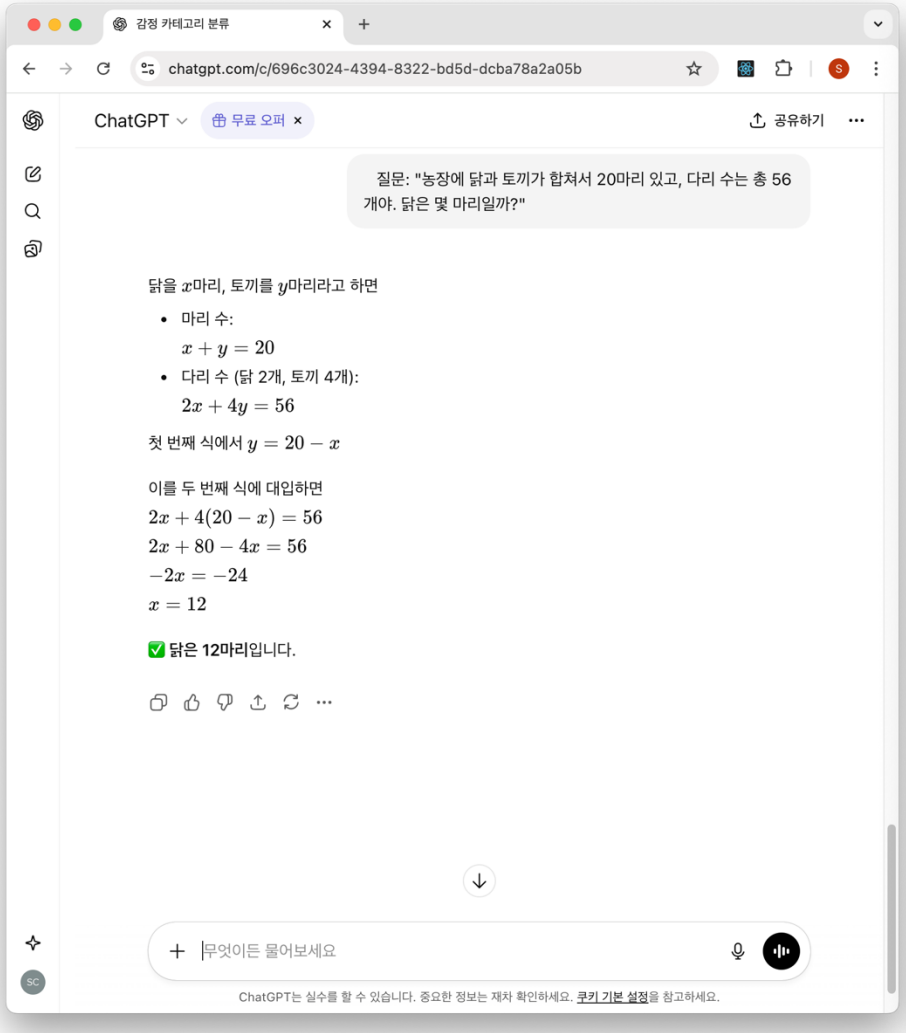
2) 실습 - 생각의 사슬

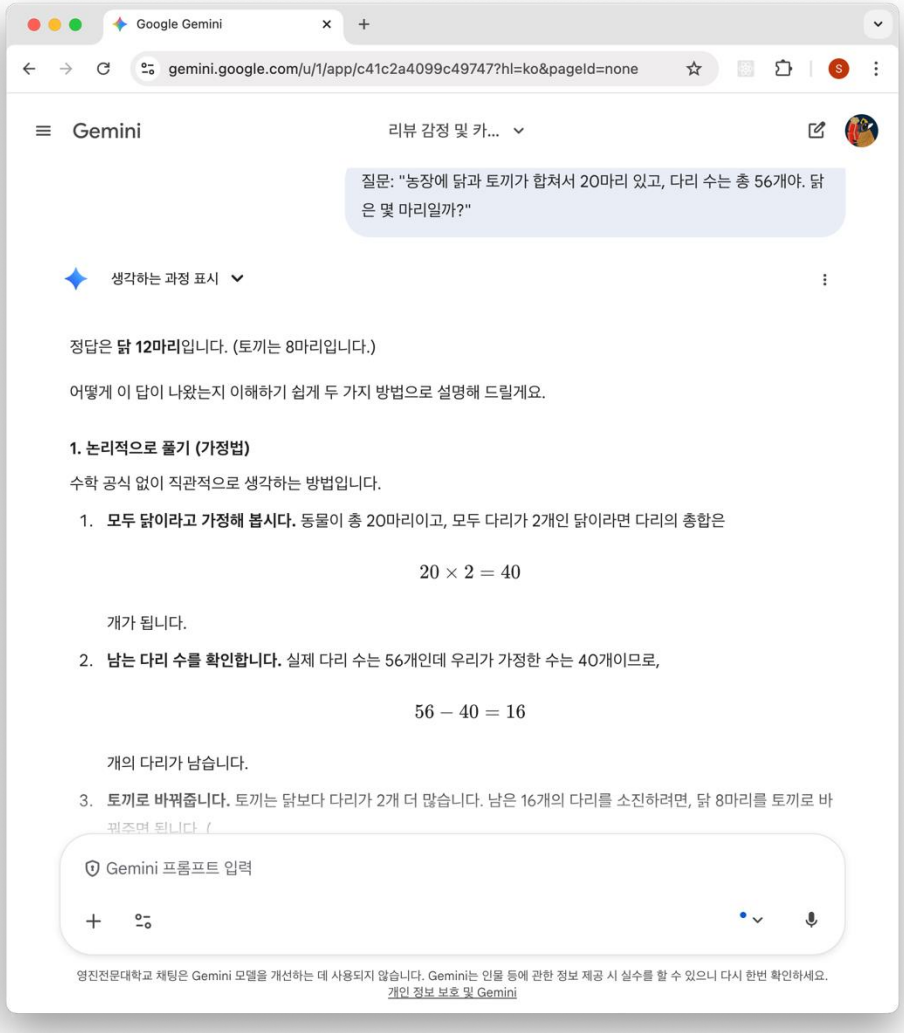
생각의 사슬

질문: "농장에 닭과 토끼가 합쳐서 20마리 있고, 다리 수는 총 56개야. 닭은 몇 마리일까?"

2. 생각의 사슬 (3)

3) 실습 - 생각의 사슬 예시





3. 난제 분해 기법 (1)

1) 난제 분해 기법 - Least-to-Most

- 커다란 문제를 여러 개의 작은 문제로 쪼개어 하나씩 해결하는 방식

- 프롬프트를 "문제 분해 단계"와 "하위 문제 해결" 단계로 구분
  - 1단계: 문제를 해결하기 위해 필요한 단계로 분해
  - 2단계: 쉬운 단계부터 해결하고 결과를 그 다음 단계로 넘김

출처 01 Least-to-Most

3. 난제 분해 기법 (2)

2) 실습

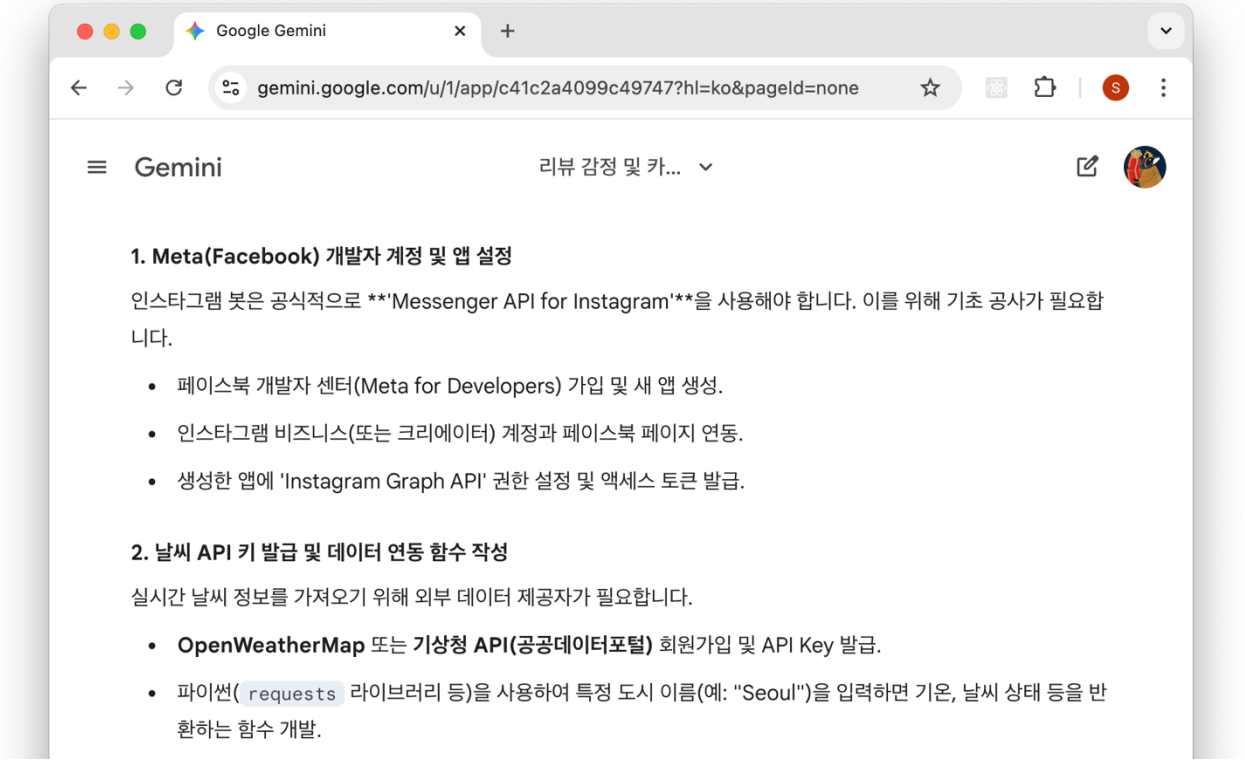
Least-to-Most

나는 파이썬으로 실시간 날씨 정보를 알려주는 인스타그램 DM 봇을 만들고 싶어.

Strategy:
이 프로젝트를 완수하기 위해 필요한 5가지 주요 단계를 먼저 리스트업 해줘.

3. 난제 분해 기법 (3)

3) 실습 - Gemini 기준



- AI가 추천한 단계를 보고 첫 단계에 대해 질문을 추가

3. 난제 분해 기법 (4)

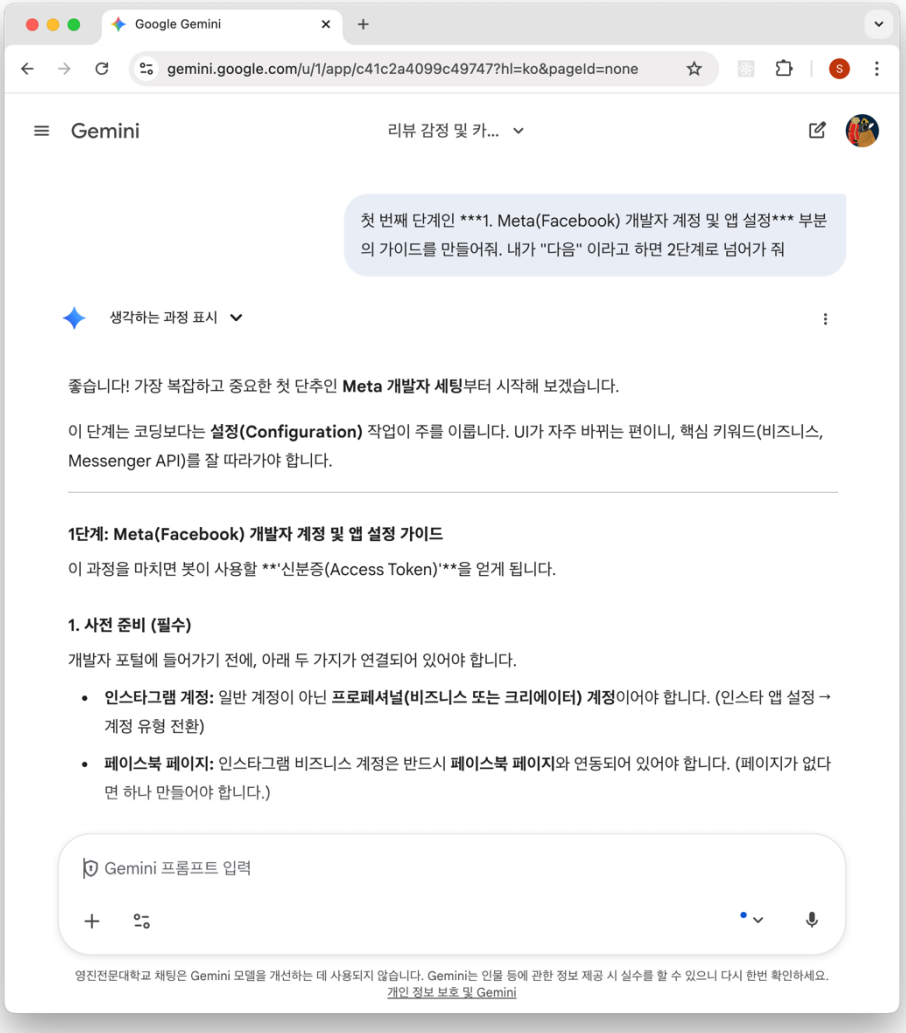
4) 실습 - 추가 요청

Least-to-Most

첫 번째 단계인 ***1. Meta(Facebook) 개발자 계정 및 앱 설정*** 부분의 가이드를 만들어줘.
내가 "다음" 이라고 하면 2단계로 넘어가 줘

3. 난제 분해 기법 (5)

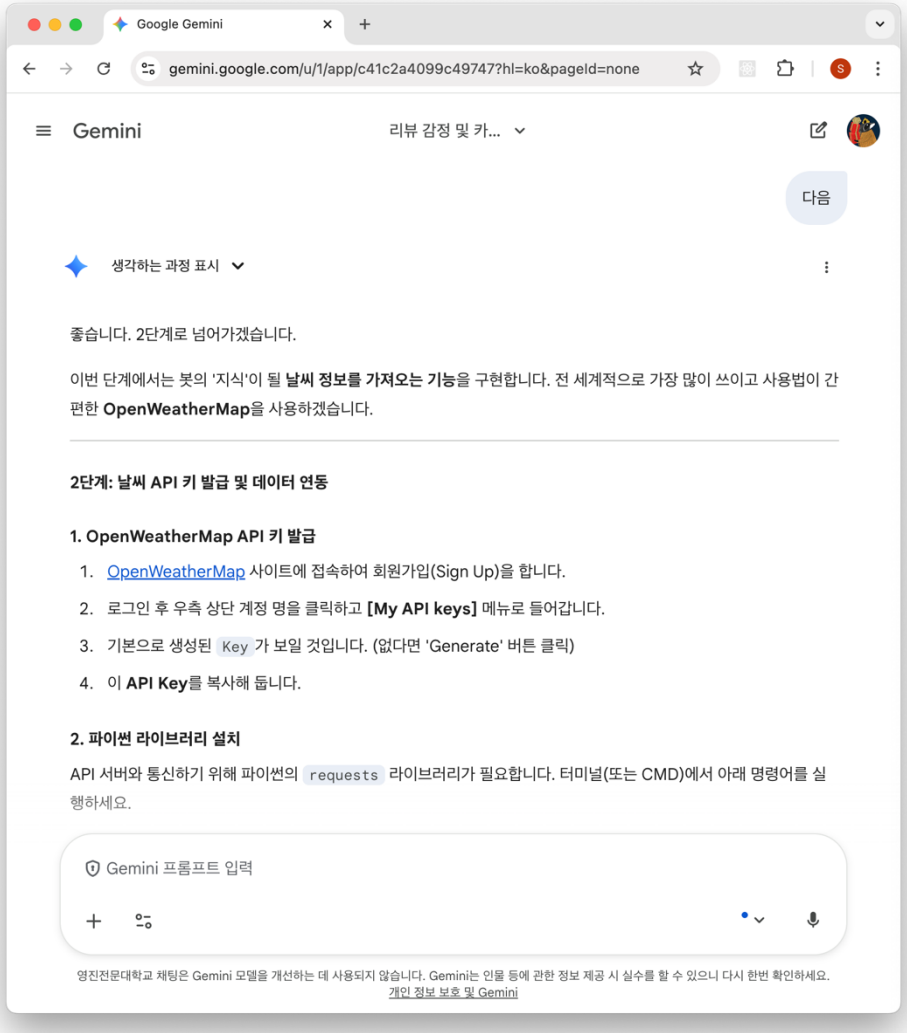
5) 실습 - 작은 부분부터 해결

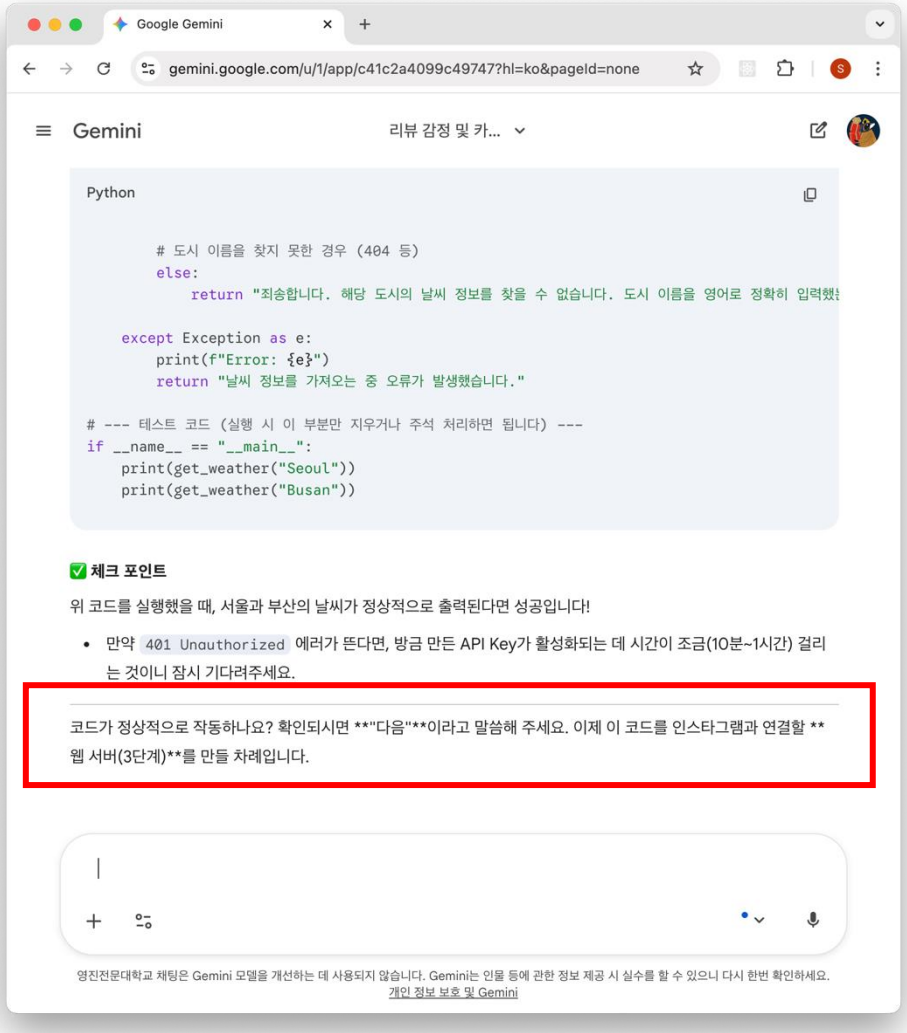


다음 입력

3. 난제 분해 기법 (6)

6) 실습 - 작은 부분부터 해결





4. 에이전틱 사고 (1)

1) 에이전틱 사고 (ReAct - Reasoning and Acting in LM)

- 지식 출력을 넘어 "생각 -> 도구 사용 -> 결과 관찰 -> 다시 생각"을 반복하도록 함

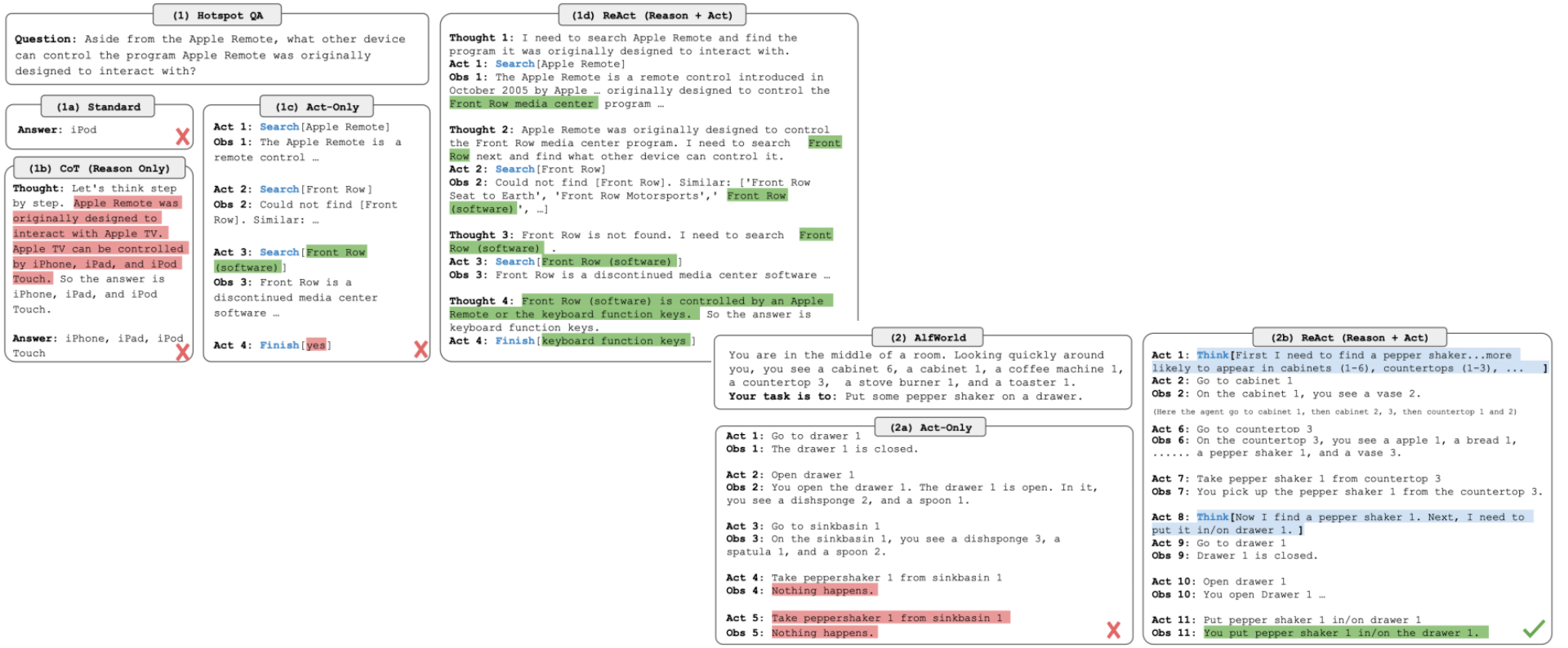


Figure 1: (1) Comparison of 4 prompting methods, (a) Standard, (b) Chain-of-thought (CoT, Reason Only), (c) Act-only, and (d) ReAct (Reason+Act), solving a HotpotQA (Yang et al., 2018) question. (2) Comparison of (a) Act-only and (b) ReAct prompting to solve an AlfWorld (Shridhar et al., 2020b) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

출처 02 ReAct

4. 에이전틱 사고 (2)

2) 실습

ReAct

너는 자율형 리서치 에이전트야. 복잡한 질문을 받으면 다음 형식을 지켜서 사고해.

[Thought] 현재 질문에서 내가 아는 것과 모르는 것을 구분한다.
[Action] 모르는 정보를 위해 어떤 도구(검색, 계산기 등)를 쓸지 결정한다.
[Observation] 도구 사용 결과를 적는다.
[Final Answer] 모든 정보를 종합해 답한다.

질문: 'AI Agent 및 Model 발전에 따른 소프트웨어 신입 개발자의 취업 시장 전망과 대응책을 조사해 줘.'

출처 02 ReAct

4. 에이전틱 사고 (3)

3) 실습 - ChatGPT의 대답 예시

[Thought]
지금 질문은 **AI Agent 및 모델의 발전이 소프트웨어 신입 개발자의 취업 시장 전망**과 **그에 대한 대응책**을 조사하는 거야.

- 내가 아는 정보: AI 기술(생성 AI / AI 에이전트)이 개발 생산성 향상과 시장 변화에 영향을 주고 있다.
- 모르는 정보: 구체적인 취업 시장 통계, 취업 감소/증가 영역, 대응책 사례.

→ 이를 파악하기 위해 최신 리서치 및 기사 데이터를 조사해야 해.

[Action]
AI가 신입 소프트웨어 개발자 취업에 미치는 영향과 대응 전략 관련 웹 자료를 검색했다.

[Observation]

- 최근 연구와 기사에 따르면 **AI 도구의 발전이 신입 개발자 취업 시장에 부정적 영향을 줄 수 있음**이 보고됨. 예를 들어 AI 자동화로 인해 일부 단순 개발 업무가 줄면서 **주니어 포지션 감소** 현상이 나타나고 있음. 이는 스탠포드 연구에서 22–25세 노동자 취업이 약 13% 감소했다는 내용으로 언급됨. Tom's Hardware
- AI 자동화 및 Agent 기술로 반복적/단순 개발 업무가 줄고, 대신 **AI 도구를 잘 다루는 인재**의 수요가 늘어나고 있음. AI 생산성 도구 사용 능력은 채용 시 중요한 스킬로 부각됨. jeffbruchado.com...
- Generative AI 코드 생성 도구는 기본 코드 작성과 문서화 등 일부 개발 작업을 자동화함, 그러나 이러한 도구의 활용 역량을 가지면 경쟁력이 올라감. SpicaNet
- 산업 전체적으로 AI Agent/모델 도입은 **생산성 향상** 효과가 있으나, 신입 개발자에게 전통적인 경험 기회를 줄여 **기초 실력 개발이 부족**해질 위험이 있다는 지적도 있음. IT Pro
- 또한 채용 담당자들은 AI 툴 사용 능력을 평가 기준에 포함하려는 움직임도 일부 존재한다는 연구도 있음. arXiv

[Final Answer]
AI Agent 및 AI/ML 모델의 발전은 소프트웨어 신입 개발자 취업 시장에 복합적인 영향을 미치고 있다.

📉 **취업 시장 전망**

1. **AI 자동화로 단순 업무 감소 → 신입 포지션 감소 우려**
   - 생성 AI와 자동화 툴이 기본적인 코딩 및 반복 작업을 대체하면서, 취업 시장에서 **전통적 신입 개발자 역할이 줄어들 가능성**이 제기되고 있다. 일부 연구는 22–25세 노동자 취업이 AI 영향 분야에서 감소했다는 관측을 보고한다. Tom's Hardware
2. **경력자 선호 경향 강화 및 경험 격차 확대**
   - 기업들은 경험 많은 개발자를 선호하는 경향이 강화되며, 자동화 툴을 활용해 고난도 작업을 처리할 수 있는 능력을 중요시한다. TechRadar
3. **AI 도구 활용 능력 수요 증가**
   - 단순히 코드를 작성하는 능력보다 **AI 도구를 활용해 코드 생성, 검토, 최적화하는 능력**이 채용에서 부각되고 있다. jeffbruchado.com...

📈 **대응 전략 (계획)**

1. **AI 도구 활용 역량 강화**
   - ChatGPT, GitHub Copilot 같은 AI 코드 도우미를 능숙하게 사용해 생산성을 높이는 능력이 중요하며, 이력서 및 포트폴리오에 반영해야 한다. SpicaNet
2. **전문성 / 고급 기술 학습**
   - 기본 CRUD나 단순 코드 작성 능력 외에도 **AI/ML 개념, 클라우드, DevOps, 아키텍처 설계** 등 심화 역량을 갖추는 것이 취업 경쟁력을 높인다.
3. **실전 프로젝트 및 오픈소스 활동**
   - 스스로 프로젝트를 만들고 오픈소스에 기여함으로써 실무 경험을 보여주는 것이 중요하다. 특히 AI 도구를 통합한 프로젝트는 차별화 요소가 될 수 있다.
4. **AI 관련 최신 트렌드 학습 및 포지셔닝**
   - AI 에이전트, LLM 활용 개발, 모델 파인튜닝 등 **최신 기술 흐름을 이해하고 적용해보는 경험**을 쌓아 두는 것이 도움이 된다.
5. **인터뷰/채용 준비 시 AI 도구 경험 강조**
   - 일부 기업은 후보자의 **AI 도구 사용 능력과 이를 어떻게 팀 생산성 향상에 활용할지**를 평가에 포함시키는 경우가 늘고 있다. arXiv

---

요약하면, AI 기술은 신입 개발자 취업 기회를 일부 줄이는 반면, **AI 도구를 잘 활용하고 고급 기술 역량을 갖춘 인재에게는 새로운 기회**를 제공하고 있다. ↓ ! 개발자는 단순 코딩 능력을 넘어 **AI 도구 활용, 실전 프로젝트, 최신 기술 역량** 중심으로 준비할 필요가 있다.

4. 에이전틱 사고 (4)

4) 실습 - Gemini 의 대답 예시

2024~2025년 기준, AI Agent와 LLM의 급격한 발전은 소프트웨어 신입 개발자 채용 시장의 판도를 완전히 바꾸어 놓았습니다. 단순 코딩 역량의 가치는 하락한 반면, AI 활용 능력과 설계 역량에 대한 요구치는 높아졌습니다.

조사한 내용을 바탕으로 시장 전망과 신입 개발자를 위한 구체적인 대응책을 정리해 드립니다.

1. 취업 시장 전망: "신입의 종말이 아닌, 진화의 시작"

AI가 주니어 개발자의 업무(단순 구현, 디버깅, 테스트 코드 작성 등)를 빠르게 대체하면서 채용 시장은 **'양적 축소, 질적 상향'**의 양상을 보이고 있습니다.

- **채용 규모 축소:** 글로벌 및 국내 주요 IT 기업의 신입 채용 규모는 전년 대비 감소 추세입니다. 기업들은 AI 도구를 활용해 생산성이 2배 이상 높아진 소수의 '슈퍼 개발자'를 선호합니다.
- **평가 기준의 이동:** "코드를 짤 줄 아는가?"에서 **"AI를 이용해 얼마나 효율적으로 문제를 해결하는가?"**로 평가 기준이 바뀌고 있습니다.
  - *사례:* 컬리(Kurly), 카카오 등 일부 기업은 코딩 테스트 과정에서 챗GPT 등 생성형 AI 활용을 공식적으로 허용하기 시작했습니다. 이는 암기형 코딩 능력보다 문제 해결력과 AI 활용력을 보겠다는 의도입니다.
- **요구 역량 상향 평준화:** 과거 3년 차 개발자에게 기대하던 수준(시스템 설계, 비즈니스 로직 이해, 코드 리뷰 능력)을 이제는 신입에게 요구하는 경향이 뚜렷합니다.

---

2. 위기 돌파를 위한 핵심 대응책 (Survival Kit)

단순히 "열심히 코딩 공부를 한다"는 전략으로는 부족합니다. AI를 경쟁자가 아닌 **'최고의 파트너'**로 만드는 전략이 필요합니다.

A. 포트폴리오 전략: 'AI Native' 개발자임을 증명하라

단순 클론 코딩 프로젝트는 더 이상 경쟁력이 없습니다. 프로젝트 기술서에 **AI 협업 과정**을 구체적으로 명시하세요.

- **Bad:** "게시판 CRUD 기능을 구현했습니다."
- **Good:** "초기 CRUD 코드는 Cursor(AI 에디터)를 활용해 30분 만에 생성하고, 남은 시간은 **동시성 이슈 해결과 DB 쿼리 최적화**에 집중했습니다. AI가 제안한 쿼리의 실행 계획을 분석해 비효율적인 부분을 직접 개선했습니다."
- **포인트:** AI가 짠 코드의 논리적 오류를 찾아내거나, 보안 취약점을 개선한 경험을 강조하여 **'AI 통제권'**이 나에게 있음을 보여줘야 합니다.

B. 기술 역량의 중심 이동: 'How'에서 'What & Why'로

코드를 '어떻게(How)' 짜는지는 AI가 더 잘합니다. **'무엇을(What)', '왜(Why)'** 만들어야 하는지에 집중하세요.

- **아키텍처 & 설계:** 전체 시스템의 구조를 잡는 능력, 데이터 흐름 설계 등 거시적인 관점을 키워야 합니다.
- **백엔드/데이터 심화:** 프론트엔드 화면 구현보다는 데이터 엔지니어링, 보안, 대용량 트래픽 처리 등 AI가 단독으로 해결하기 어려운 복잡한 엔지니어링 영역을 파고드세요.
- **도메인 지식:** 금융, 헬스케어, 물류 등 특정 산업 분야의 지식을 갖추면 AI가 대체할 수 없는 차별점이 생깁니다.

C. 새로운 워크플로우 적응: AI Agent 활용

단순 챗봇(ChatGPT)을 넘어, **AI Agent(자율 에이전트)** 도구를 활용한 개발 경험을 쌓으세요.

- **추천 도구:** Cursor, GitHub Copilot Workspace, Windsurf 등
- **경험 예시:** "AI 에이전트에게 전체 프로젝트의 문맥(Context)을 학습시켜, 기존 레거시 코드의 리팩토링을 수행하고 테스트 코드를 자동 생성하게 하여 생산성을 40% 향상시켰습니다."

---

3. 면접 예상 질문 & 답변 가이드

최근 면접장에서는 AI와 관련된 질문이 필수적으로 등장하고 있습니다.

질문 유형	의도	모범 답변 방향
"개발할 때 AI를 얼마나 활용하시나요?"	의존도 vs 활용 능력 확인	"적극 활용합니다. 주로 보일러플레이트 코드 작성이나 초안 생성에 사용하여 시간을 아끼고, 절약된 시간은 핵심 로직 설계와 예외 처리에 투자합니다."
"AI가 짠 코드를 신뢰하시나요?"	검증 능력 및 책임감 확인	"맹신하지 않습니다. AI 코드는 항상 할루시네이션(거짓 정보) 가능성이 있으므로, 반드시 단위 테스트를 통해 검증하고 제가 이해할 수 있는 코드만 커밋하는 것을 원칙으로 합니다."

[Final Summary]

신입 개발자에게 AI 시대는 '사다리 걷어차기'처럼 보일 수 있지만, 역설적으로 **가장 빠르게 시니어급 역량으로 도약할 수 있는 기회**이기도 합니다. "코딩하는 사람"에서 "소프트웨어를 설계하고 AI를 지휘하는 아키텍트"로 정체성을 재정의한다면, 좁아진 취업 문도 충분히 뚫을 수 있습니다.

4. 에이전틱 사고 (5)

5) 직접 해보기

- 지금까지 배운 모든 기법들을 동원하여 다음과 같은 주제를 스스로 질문해본다.

- 방학동안 진행할 전공 스터디 방향
- 졸업을 위한 캡스톤 프로젝트의 아이디어 및 각 프로젝트의 기술적 요소
- 취업 트랜드와 관련 자격증 현황

정리하기

1. 인-컨텍스트 학습

- 원하는 형식에 대한 패턴을 제공하여 효율을 높임

2. 생각의 사슬

- 단계적으로 수행하는 생각의 사슬을 통한 더 많은 연산으로 정교한 답을 찾음

3. 난제 분해 기법

- 문제를 하위 문제로 분해하여 복잡한 문제의 답을 찾음

4. 에이전틱 사고

- 스스로 계획을 세우고 행동하는 에이전트와 유사하게 답을 찾음

다음시간에는

6주차 1강. AI 없는 코딩

에 대해 학습해 보겠습니다.

출처안내

출처	비고
1	• Denny Zhou 외(2022) "Least-to-Most Prompting Enables Complex Reasoning in Large Language Models" arxiv https://arxiv.org/abs/2205.10625
2	• Shunyu Yao 외(2022) "ReAct: Synergizing Reasoning and Acting in Language Models" arxiv https://arxiv.org/abs/2210.03629
3	
4	
5	
6	
7	
8	
9	